Tab 1

🚀 Team Onboarding Guide: Local Setup & Feature Development

Welcome to the **Research AI Platform** project! Follow these steps in order to get the project running on your machine and start building your assigned features.

📋 Phase 1: Machine Prerequisites

Make sure you have the following installed on your computer before starting:

1. **Python 3.13.x** — Download Python 3.13 *(⚠️ Check "Add python.exe to PATH" during installation!)*
2. **Git** — Download Git
3. **PostgreSQL 16 or 17** — Download PostgreSQL *(Remember the superuser password you set during installation)*

📁 Phase 2: Clone & Environment Setup

Open **PowerShell** in the directory where you want your project folder to live, and run:

1. Clone the GitHub Repository

PowerShell
```
git clone https://github.com/rafitboo/ai-research-assistant.git
cd ai-research-assistant
```

2. Create & Activate Virtual Environment

PowerShell
```
python -m venv venv  // ei line e  py -3.13 -m venv venv
.\venv\Scripts\Activate.ps1
```

(You should now see `(venv)` at the left of your terminal prompt).

3. Install Required Dependencies

PowerShell
```
pip install -r requirements.txt
pip install email-validator --upgrade
```

pip install sqlalchemy --upgrade

🛢️ Phase 3: Local PostgreSQL Database Setup

1. Create User and Database

Open the app→ **SQL Shell (psql)** from your Windows Start Menu (press Enter for defaults and enter your root PostgreSQL password when prompted), then execute these SQL commands:

SQL
CREATE USER research_user WITH PASSWORD '471';
CREATE DATABASE research_db OWNER research_user;
GRANT ALL PRIVILEGES ON DATABASE research_db TO research_user;

2. Create the `.env` File

In the project root, navigate into the `backend` directory and create a new file named `.env`:

File Path: `ai-research-assistant/backend/.env`

Code snippet
DB_USER=research_user
DB_PASSWORD=my_secure_password
DB_HOST=127.0.0.1
DB_PORT=5432
DB_NAME=research_db
DATABASE_URL=postgresql://research_user:my_secure_password@127.0.0.1:5432/research_db

🧪 Phase 4: Run the Application Locally

You will need **two terminal windows** running simultaneously:

Terminal 1: FastAPI Backend

PowerShell
Make sure (venv) is active
cd backend
uvicorn main:app --reload --port 8000

> **Note:** On first startup, SQLAlchemy will automatically detect your local PostgreSQL database and generate all necessary tables (`users`, `papers`, `projects`, `project_members`).

Terminal 2: Flask Frontend

Open a new PowerShell window in the project root directory:

PowerShell
.\venv\Scripts\Activate.ps1
python frontend/app.py

Open your browser and visit **`[http://127.0.0.1:5000](http://127.0.0.1:5000)`**. You can now register a new account and test the **Dashboard Overview**!

Tab 2

🔄 Git Collaboration Protocol

To avoid overwriting each other's work while developing:

1. Pull Latest Changes Before Starting

PowerShell
git checkout main
git pull origin main

2. Create a Feature Branch

PowerShell
For Feature 2:
git checkout -b feature-2-paper-library

For Feature 3:
git checkout -b feature-3-progress-tracker

For Feature 4:
git checkout -b feature-4-project-creation

3. Commit & Push Your Feature

Once your assigned feature is complete and tested locally:

PowerShell
git add .
git commit -m "Completed Module 1 - Feature X: <Feature Name>"
git push origin feature-X-branch-name

Then open GitHub to merge your feature branch into `main`!

Tab 3

DB:
PUSH:
$env:Path += ";C:\Program Files\PostgreSQL\17\bin"

& "C:\Program Files\PostgreSQL\17\bin\pg_dump.exe" -U research_user -h 127.0.0.1 -p 5432 -F c -f "$HOME\research_db.backup" research_db

IF DB NOT CREATED:

- Open **SQL Shell (psql)** from the Windows Start Menu.
    -
    - Press **Enter** for these defaults:
    -
    - Server [localhost]:
    - Database [postgres]:
    - Port [5432]:
    - Username [postgres]:
    -
    - Enter your PostgreSQL administrator password. **471**
    -
    - **You should see:**
        - **postgres=#**
- CREATE DATABASE research_db OWNER research_user;
- Run this in PowerShell:
    - & "C:\Program Files\PostgreSQL\17\bin\pg_restore.exe" -h 127.0.0.1 -U research_user -d research_db --no-owner --no-privileges "C:\Users\user\Downloads\research_db.backup"
-
- Enter the password: **471**
- Run this command:

  & "C:\Program Files\PostgreSQL\17\bin\psql.exe" -h 127.0.0.1 -U research_user -d research_db\

- Enter 471
- Then run \dt
- You should see tables such as:users, papers, projects, project_members
- To inspect the data: SELECT * FROM users;
- Exit with \q

IF DB CREATED:

- Open **SQL Shell (psql)** from the Windows Start Menu.
    - Press **Enter** for these defaults:
    - Server [localhost]:
    - Database [postgres]:
    - Port [5432]:
    - Username [postgres]:
    - Enter your PostgreSQL administrator password. **471**
    - **You should see:**
        - **postgres=#**

- Run these commands one at a time:
    - SELECT pg_terminate_backend(pid)
    - FROM pg_stat_activity
    - WHERE datname = 'research_db'
    - AND pid <> pg_backend_pid();
- DROP DATABASE research_db;
- CREATE DATABASE research_db OWNER research_user;
- Run this in PowerShell:

  & "C:\Program Files\PostgreSQL\17\bin\pg_restore.exe" -h 127.0.0.1 -U research_user -d research_db --no-owner --no-privileges "C:\Users\user\Downloads\research_db.backup"
-
- Enter the password: **471**
- Run this command:

  & "C:\Program Files\PostgreSQL\17\bin\psql.exe" -h 127.0.0.1 -U research_user -d research_db\

- Enter 471
- Then run \dt

- You should see tables such as:users, papers, projects, project_members
- To inspect the data: SELECT * FROM users;
- Exit with \q

module-1 feature-2

Here is the complete implementation for **Module 1 — Feature 2: Paper Upload & Library**. Before starting, make sure you have `python-multipart` installed in your virtual environment (required by FastAPI to process PDF file uploads and form inputs):

PowerShell

```
pip install python-multipart
```

📄 File 1: Create Backend Router (New File)

Create a brand new file at `backend/app/routers/papers.py`.

Python

```
import os
import uuid
from typing import Optional
from fastapi import APIRouter, Depends, HTTPException, UploadFile, File, Form, Query
from fastapi.responses import FileResponse
from sqlalchemy.orm import Session
from app.database import get_db
from app.models import Paper, User
from app.auth_utils import get_current_user

router = APIRouter(prefix="/api/papers", tags=["Papers"])

# Upload directory path inside backend folder
UPLOAD_DIR = os.path.join(os.path.dirname(os.path.dirname(os.path.dirname(__file__))),
"uploads")
os.makedirs(UPLOAD_DIR, exist_ok=True)


@router.post("/upload")
def upload_paper(
    title: str = Form(...),
    author: Optional[str] = Form(None),
    year: Optional[int] = Form(None),
    topic: Optional[str] = Form(None),
    research_area: Optional[str] = Form(None),
    tags: Optional[str] = Form(None),
    file: Optional[UploadFile] = File(None),
    current_user: dict = Depends(get_current_user),
    db: Session = Depends(get_db)
):
    user_id = current_user["user_id"]

    saved_file_path = None
    if file and file.filename:
        # Generate unique filename to prevent overwriting
        file_ext = os.path.splitext(file.filename)[1]
```

```
        unique_filename = f"{uuid.uuid4().hex}{file_ext}"
        saved_file_path = os.path.join(UPLOAD_DIR, unique_filename)

        with open(saved_file_path, "wb") as buffer:
            buffer.write(file.file.read())

    new_paper = Paper(
        title=title,
        author=author,
        year=year,
        topic=topic,
        research_area=research_area,
        tags=tags,
        file_path=saved_file_path,
        reading_status="Unread",
        read_percentage=0,
        time_spent_seconds=0,
        user_id=user_id
    )
    db.add(new_paper)
    db.commit()
    db.refresh(new_paper)

    return {"message": "Paper uploaded successfully", "paper_id": new_paper.id}


@router.get("/")
def list_papers(
    topic: Optional[str] = Query(None),
    year: Optional[int] = Query(None),
    author: Optional[str] = Query(None),
    research_area: Optional[str] = Query(None),
    search: Optional[str] = Query(None),
    current_user: dict = Depends(get_current_user),
    db: Session = Depends(get_db)
):
    user_id = current_user["user_id"]
    query = db.query(Paper).filter(Paper.user_id == user_id)

    # Filtering logic
    if topic and topic.strip():
        query = query.filter(Paper.topic.ilike(f"%{topic.strip()}%"))
    if year:
        query = query.filter(Paper.year == year)
    if author and author.strip():
        query = query.filter(Paper.author.ilike(f"%{author.strip()}%"))
```

```
    if research_area and research_area.strip():
        query = query.filter(Paper.research_area.ilike(f"%{research_area.strip()}%"))
    if search and search.strip():
        search_filter = f"%{search.strip()}%"
        query = query.filter(
            (Paper.title.ilike(search_filter)) |
            (Paper.tags.ilike(search_filter)) |
            (Paper.author.ilike(search_filter))
        )

    papers = query.order_by(Paper.created_at.desc()).all()
    return [
        {
            "id": p.id,
            "title": p.title,
            "author": p.author,
            "year": p.year,
            "topic": p.topic,
            "research_area": p.research_area,
            "tags": p.tags,
            "file_path": p.file_path,
            "reading_status": p.reading_status,
            "read_percentage": p.read_percentage,
            "time_spent_seconds": p.time_spent_seconds,
            "created_at": p.created_at.isoformat() if p.created_at else None
        }
        for p in papers
    ]


@router.get("/{paper_id}/file")
def download_paper_file(
    paper_id: int,
    current_user: dict = Depends(get_current_user),
    db: Session = Depends(get_db)
):
    user_id = current_user["user_id"]
    paper = db.query(Paper).filter(Paper.id == paper_id, Paper.user_id == user_id).first()
    if not paper or not paper.file_path:
        raise HTTPException(status_code=404, detail="Paper or file not found")

    if not os.path.exists(paper.file_path):
        raise HTTPException(status_code=404, detail="File missing on disk")

    return FileResponse(paper.file_path, media_type="application/pdf", filename=f"{paper.title}.pdf")
```

🛠️ File 2: Modify `backend/main.py`

Open `backend/main.py` and make only two targeted edits:

Edit 1: Update Router Imports

Locate line 4 in `backend/main.py`:

Python

```
# CHANGE THIS LINE:
from app.routers import auth, admin, dashboard

# TO THIS:
from app.routers import auth, admin, dashboard, papers
```

Edit 2: Register Papers Router

Locate where the routers are included (around lines 17-20):

Python

```
app.include_router(auth.router)
app.include_router(admin.router)
app.include_router(dashboard.router)

# ADD THIS LINE RIGHT AFTER dashboard.router:
app.include_router(papers.router)
```

🛠️ File 3: Modify `frontend/app.py`

Open `frontend/app.py` and append these three route functions at the **very end** of the file:

Python

```
# ========================================
# Module 1 - Feature 2: Paper Library Routes
# ========================================

@app.route("/papers")
def papers_library():
    if "token" not in session:
        return redirect(url_for("login"))

    headers = {"Authorization": f"Bearer {session['token']}"}
    params = {}
    for key in ["topic", "year", "author", "research_area", "search"]:
        val = request.args.get(key)
        if val:
            params[key] = val

    papers = []
    try:
        response = httpx.get(f"{API_BASE_URL}/papers/", headers=headers, params=params)
        if response.status_code == 200:
```

```
            papers = response.json()
    except Exception as e:
        flash(f"Error fetching papers: {str(e)}", "error")

    return render_template("papers.html", user=session.get("user"), papers=papers,
filters=request.args)


@app.route("/papers/upload", methods=["POST"])
def upload_paper_route():
    if "token" not in session:
        return redirect(url_for("login"))

    headers = {"Authorization": f"Bearer {session['token']}"}

    data = {
        "title": request.form.get("title"),
        "author": request.form.get("author"),
        "year": request.form.get("year") or None,
        "topic": request.form.get("topic"),
        "research_area": request.form.get("research_area"),
        "tags": request.form.get("tags")
    }

    files = None
    if "file" in request.files and request.files["file"].filename != "":
        file_obj = request.files["file"]
        files = {"file": (file_obj.filename, file_obj.stream.read(), file_obj.content_type)}

    try:
        response = httpx.post(f"{API_BASE_URL}/papers/upload", headers=headers, data=data,
files=files)
        if response.status_code == 200:
            flash("Paper uploaded successfully to your library!", "success")
        else:
            err = response.json().get("detail", "Upload failed")
            flash(f"Upload failed: {err}", "error")
    except Exception as e:
        flash(f"Error uploading paper: {str(e)}", "error")

    return redirect(url_for("papers_library"))


@app.route("/papers/<int:paper_id>/file")
def download_paper(paper_id):
    if "token" not in session:
```

```
        return redirect(url_for("login"))

    headers = {"Authorization": f"Bearer {session['token']}"}
    try:
        backend_res = httpx.get(f"{API_BASE_URL}/papers/{paper_id}/file", headers=headers)
        if backend_res.status_code == 200:
            from flask import Response
            return Response(
                backend_res.content,
                mimetype="application/pdf",
                headers={"Content-Disposition": f"inline; filename=paper_{paper_id}.pdf"}
            )
        else:
            flash("PDF file not found or unavailable.", "error")
    except Exception as e:
        flash(f"Error: {str(e)}", "error")

    return redirect(url_for("papers_library"))
```

🎨 File 4: Create Frontend View (New File)

Create a brand new template file at `frontend/templates/papers.html`.

HTML

```
{% extends "base.html" %}
{% block content %}
<div class="space-y-6 max-w-7xl mx-auto">
    <!-- Header with Modal Toggle -->
    <div class="flex items-center justify-between">
        <div>
            <h1 class="text-3xl font-extrabold text-slate-900">Paper Library</h1>
            <p class="text-slate-600 text-sm mt-1">Upload, search, tag, and organize your research papers.</p>
        </div>
        <button onclick="document.getElementById('uploadModal').classList.remove('hidden')" class="px-5 py-2.5 bg-[#5B3DF5] hover:bg-[#4A3AFF] text-white font-bold rounded-xl shadow-sm transition flex items-center gap-2 text-sm">
            <span>📤</span> Upload Paper
        </button>
    </div>

    <!-- Filter & Search Bar -->
    <div class="bg-white p-5 rounded-2xl border border-slate-200 shadow-sm space-y-4">
        <form method="GET" action="/papers" class="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-6 gap-3">
            <input type="text" name="search" value="{{ filters.get('search', '') }}" placeholder="Search title, tag, or author..." class="lg:col-span-2 px-4 py-2 border border-slate-300 rounded-xl text-sm focus:outline-none focus:ring-2 focus:ring-indigo-500">
```

```
            <input type="text" name="topic" value="{{ filters.get('topic', '') }}" placeholder="Topic (e.g. Vision)" class="px-4 py-2 border border-slate-300 rounded-xl text-sm focus:outline-none focus:ring-2 focus:ring-indigo-500">
            <input type="text" name="author" value="{{ filters.get('author', '') }}" placeholder="Author" class="px-4 py-2 border border-slate-300 rounded-xl text-sm focus:outline-none focus:ring-2 focus:ring-indigo-500">
            <input type="number" name="year" value="{{ filters.get('year', '') }}" placeholder="Year (e.g. 2025)" class="px-4 py-2 border border-slate-300 rounded-xl text-sm focus:outline-none focus:ring-2 focus:ring-indigo-500">
            <div class="flex gap-2">
                <button type="submit" class="w-full bg-slate-800 text-white font-bold text-sm py-2 rounded-xl hover:bg-slate-900 transition">Filter</button>
                <a href="/papers" class="px-3 py-2 bg-slate-100 text-slate-600 font-bold text-sm rounded-xl hover:bg-slate-200 flex items-center justify-center">Reset</a>
            </div>
        </form>
    </div>

    <!-- Papers Grid -->
    <div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-5">
        {% if papers %}
            {% for paper in papers %}
            <div class="bg-white p-6 rounded-2xl border border-slate-200 shadow-sm hover:shadow-md transition flex flex-col justify-between">
                <div class="space-y-3">
                    <div class="flex items-center justify-between gap-2">
                        <span class="px-2.5 py-1 bg-indigo-50 text-indigo-700 text-xs font-bold rounded-lg border border-indigo-100">
                            {{ paper.topic or 'General' }}
                        </span>
                        {% if paper.year %}
                        <span class="text-xs font-semibold text-slate-400">{{ paper.year }}</span>
                        {% endif %}
                    </div>

                    <h3 class="font-bold text-slate-900 text-base leading-snug line-clamp-2" title="{{ paper.title }}">
                        {{ paper.title }}
                    </h3>

                    {% if paper.author %}
                    <p class="text-xs text-slate-500 font-medium">👤 {{ paper.author }}</p>
                    {% endif %}

                    {% if paper.research_area %}
                    <p class="text-xs text-slate-500 font-medium">🔬 Area: {{ paper.research_area }}</p>
```

```
                {% endif %}

                <!-- Tags List -->
                {% if paper.tags %}
                <div class="flex flex-wrap gap-1.5 pt-1">
                   {% for tag in paper.tags.split(',') %}
                   <span class="px-2 py-0.5 bg-slate-100 text-slate-600 text-xs rounded-md
font-medium">
                      #{{ tag.strip() }}
                   </span>
                   {% endfor %}
                </div>
                {% endif %}
             </div>

             <div class="pt-5 mt-4 border-t border-slate-100 flex items-center justify-between">
                <span class="text-xs font-bold px-2.5 py-1 rounded-full {% if paper.reading_status ==
'Completed' %}bg-emerald-100 text-emerald-800{% elif paper.reading_status == 'Reading'
%}bg-amber-100 text-amber-800{% else %}bg-slate-100 text-slate-600{% endif %}">
                   {{ paper.reading_status }}
                </span>

                {% if paper.file_path %}
                <a href="/papers/{{ paper.id }}/file" target="_blank" class="text-xs font-bold
text-indigo-600 hover:underline flex items-center gap-1">
                   View PDF ↗
                </a>
                {% else %}
                <span class="text-xs text-slate-400 font-medium">No PDFAttached</span>
                {% endif %}
             </div>
          </div>
          {% endfor %}
       {% else %}
          <div class="col-span-full bg-white p-12 text-center rounded-2xl border border-slate-200
text-slate-500 font-medium space-y-2">
             <p class="text-3xl">📚</p>
             <p>No papers found matching your criteria.</p>
             <p class="text-xs text-slate-400">Click "Upload Paper" above to add research PDFs to
your library.</p>
          </div>
       {% endif %}
    </div>
</div>

<!-- Upload Modal -->
```

```
<div id="uploadModal" class="hidden fixed inset-0 bg-slate-900/50 backdrop-blur-sm flex items-center justify-center p-4 z-50">
  <div class="bg-white rounded-2xl max-w-lg w-full p-6 shadow-xl border border-slate-200 space-y-5">
    <div class="flex items-center justify-between border-b border-slate-100 pb-3">
      <h2 class="text-lg font-bold text-slate-900">Upload Paper to Library</h2>
      <button onclick="document.getElementById('uploadModal').classList.add('hidden')" class="text-slate-400 hover:text-slate-600 font-bold">✕</button>
    </div>

    <form method="POST" action="/papers/upload" enctype="multipart/form-data" class="space-y-4">
      <div>
        <label class="block text-xs font-bold text-slate-700 uppercase mb-1">Paper Title *</label>
        <input type="text" name="title" required placeholder="Enter research paper title" class="w-full px-4 py-2 border border-slate-300 rounded-xl text-sm focus:outline-none focus:ring-2 focus:ring-indigo-500">
      </div>

      <div class="grid grid-cols-2 gap-3">
        <div>
          <label class="block text-xs font-bold text-slate-700 uppercase mb-1">Author(s)</label>
          <input type="text" name="author" placeholder="e.g. Vaswani et al." class="w-full px-4 py-2 border border-slate-300 rounded-xl text-sm focus:outline-none focus:ring-2 focus:ring-indigo-500">
        </div>
        <div>
          <label class="block text-xs font-bold text-slate-700 uppercase mb-1">Year</label>
          <input type="number" name="year" placeholder="2025" class="w-full px-4 py-2 border border-slate-300 rounded-xl text-sm focus:outline-none focus:ring-2 focus:ring-indigo-500">
        </div>
      </div>

      <div class="grid grid-cols-2 gap-3">
        <div>
          <label class="block text-xs font-bold text-slate-700 uppercase mb-1">Topic Tag</label>
          <input type="text" name="topic" placeholder="e.g. Vision, FL, NLP" class="w-full px-4 py-2 border border-slate-300 rounded-xl text-sm focus:outline-none focus:ring-2 focus:ring-indigo-500">
        </div>
        <div>
          <label class="block text-xs font-bold text-slate-700 uppercase mb-1">Research Area</label>
          <input type="text" name="research_area" placeholder="e.g. Machine Learning" class="w-full px-4 py-2 border border-slate-300 rounded-xl text-sm focus:outline-none focus:ring-2 focus:ring-indigo-500">
```

```
            </div>
        </div>

        <div>
            <label class="block text-xs font-bold text-slate-700 uppercase mb-1">Custom Tags (Comma Separated)</label>
            <input type="text" name="tags" placeholder="e.g. Transformer, Attention, Thesis" class="w-full px-4 py-2 border border-slate-300 rounded-xl text-sm focus:outline-none focus:ring-2 focus:ring-indigo-500">
        </div>

        <div>
            <label class="block text-xs font-bold text-slate-700 uppercase mb-1">PDF File</label>
            <input type="file" name="file" accept=".pdf" class="w-full text-sm text-slate-500 file:mr-4 file:py-2 file:px-4 file:rounded-xl file:border-0 file:text-xs file:font-bold file:bg-indigo-50 file:text-indigo-700 hover:file:bg-indigo-100">
        </div>

        <div class="flex justify-end gap-3 pt-3 border-t border-slate-100">
            <button type="button" onclick="document.getElementById('uploadModal').classList.add('hidden')" class="px-4 py-2 text-sm font-bold text-slate-600 bg-slate-100 rounded-xl hover:bg-slate-200">Cancel</button>
            <button type="submit" class="px-5 py-2 text-sm font-bold text-white bg-[#5B3DF5] rounded-xl hover:bg-[#4A3AFF]">Upload Paper</button>
        </div>
      </form>
   </div>
</div>
{% endblock %}
```

🧪 Testing & Git Commit

1. Restart your FastAPI backend server:
2. PowerShell

```
uvicorn main:app --reload --port 8000
```

3.
4.
5. Navigate to http://127.0.0.1:5000/papers in your browser.
6. Test uploading a PDF paper and filtering by topic, year, or custom tags!
7. Commit and push cleanly:
8. PowerShell

```
git add .
git commit -m "Module 1 - Feature 2: Paper Upload & Library implementation"
git push origin main
```

9.

10.